Lesson 7: Introduction to Data Visualization

Python Essentials — Code the Dream

Game Plan for Today

Section	Time	What We'll Do
Warm-Up	5 min	Match the chart to the question
I Do	10–15 min	Matplotlib basics, Seaborn, customization
We Do	15–20 min	Build visualizations from a dataset together
You Do	5–10 min	Create a chart from a prompt
Wrap-Up	5 min	Assignment preview & capstone viz goals

Warm-Up (5 min)

Match the Chart to the Question

Question	Chart Type
How has revenue changed month to month?	???
Which region has the highest sales?	???
What's the distribution of customer ages?	???
Which features are correlated?	???

Options: Line Plot, Bar Plot, Histogram, Heatmap

Share your matches in the chat.

I Do: Why Visualize?

> A good chart can tell a story that a table of numbers can't.

Visualization helps you:

- **Spot patterns** (trends, clusters, outliers)
- **Compare** categories and groups
- **Communicate** findings to non-technical audiences

Two main libraries:

Library	Strengths
Matplotlib	Full control, basic plots, highly customizable
Seaborn	Statistical plots, nicer defaults, works with DataFrames

```
import matplotlib.pyplot as plt
import seaborn as sns
```

I Do: Matplotlib — Line Plot

Line plots show trends over continuous data:

```
import matplotlib.pyplot as plt

months = ["Jan", "Feb", "Mar", "Apr", "May", "Jun"]
revenue = [1000, 1200, 1500, 1700, 1600, 1800]

plt.plot(months, revenue, marker="o", color="blue")
plt.title("Monthly Revenue")
plt.xlabel("Month")
plt.ylabel("Revenue ($)")
plt.show()
```

Key elements:

- `marker="o"` adds dots at data points
- Always add `title`, `xlabel`, `ylabel`

> Every plot needs a title and axis labels — unlabeled charts are useless

I Do: Matplotlib — Bar Plot & Histogram

Bar plot — comparing categories:

```
regions = ["North", "South", "East", "West"]
sales = [500, 700, 600, 800]

plt.bar(regions, sales, color=["skyblue", "salmon",
                               "lightgreen", "gold"])
plt.title("Sales by Region")
plt.xlabel("Region")
plt.ylabel("Sales ($)")
plt.show()
```

I Do: Matplotlib — Bar Plot & Histogram (Cont.)

Histogram — distribution of values:

```
import numpy as np

data = np.random.randn(1000)
plt.hist(data, bins=30, color="purple", alpha=0.7)
plt.title("Distribution of Random Data")
plt.xlabel("Value")
plt.ylabel("Frequency")
plt.show()
```

I Do: Customizing Plots

Make plots more informative:

```
plt.plot(months, revenue, marker="o", linestyle="--",
         color="red", linewidth=2)
plt.title("Monthly Revenue", fontsize=14, fontweight="bold")
plt.xlabel("Month", fontsize=12)
plt.ylabel("Revenue ($)", fontsize=12)
plt.grid(color="gray", linestyle="--", linewidth=0.5)
plt.legend(["Revenue"], loc="upper left")
plt.show()
```

I Do: Customizing Plots (Cont.)

Common customizations:

Parameter	What It Does
`linestyle`	`"-"`, `"--"`, `":"`, `"-."`
`linewidth`	Thickness of the line
`fontsize`	Size of text labels
`grid()`	Background gridlines
`legend()`	Label for each data series

I Do: Seaborn — Heatmaps

Heatmaps visualize correlations between columns:

```
import seaborn as sns

# Compute correlations
corr = df.corr(numeric_only=True)

# Plot as heatmap
plt.figure(figsize=(10, 6))
sns.heatmap(corr, annot=True, cmap="coolwarm", fmt=".2f")
plt.title("Correlation Heatmap")
plt.show()
```

I Do: Seaborn — Heatmaps (Cont.)

Reading a heatmap:

- Values range from **-1.0** to **1.0**
- **Positive** (warm/red): both go up together
- **Negative** (cool/blue): one goes up, the other goes down
- **Near 0**: no relationship

I Do: Seaborn — Pair Plots

Pair plots show relationships between multiple variables:

```
sns.pairplot(df, vars=["age", "fare"],
             hue="survived", palette="Set2")
plt.show()
```

What you see:

- **Diagonal:** Distribution of each variable (KDE plot)
- **Off-diagonal:** Scatter plots between variable pairs
- **Colors:** Groups defined by `hue`

> Pair plots are great for exploratory analysis — they show you which variables relate to each other at a glance.

I Do: Subplots — Multiple Charts in One Figure

```
fig, axes = plt.subplots(1, 2, figsize=(12, 5))

# Left: bar plot
axes[0].bar(["A", "B", "C"], [10, 20, 15])
axes[0].set_title("Bar Plot")

# Right: histogram
axes[1].hist(np.random.randn(500), bins=20, color="teal")
axes[1].set_title("Histogram")

plt.tight_layout()
plt.show()
```

I Do: Subplots — Multiple Charts in One Figure (Cont.)

Key details:

- `plt.subplots(rows, cols)` creates a grid
- Access each subplot with `axes[index]`
- Use `set_title()` (not `plt.title()`) on each subplot
- `tight_layout()` prevents overlapping labels

We Do: Visualizing the Titanic Dataset

Let's build some charts together using Seaborn's built-in data:

```
import seaborn as sns
import matplotlib.pyplot as plt

titanic = sns.load_dataset("titanic")
print(titanic.head())
print(titanic.columns.tolist())
```

Discussion: What columns do we have to work with?

> Key columns: `survived`, `pclass`, `age`, `fare`, `sex`, `alone`

We Do: Histogram — Age Distribution

```
plt.hist(titanic["age"].dropna(), bins=25,
         color="steelblue", edgecolor="black")
plt.title("Age Distribution of Titanic Passengers")
plt.xlabel("Age")
plt.ylabel("Count")
plt.show()
```

Discussion: What does this tell us?

- Where is the peak?
- Are there many children? Elderly passengers?

> `.dropna()` removes NaN values — without it, the histogram may error.

We Do: Heatmap — What Correlates with Survival?

```
corr = titanic.corr(numeric_only=True)

plt.figure(figsize=(10, 6))
sns.heatmap(corr, annot=True, cmap="coolwarm", fmt=".2f")
plt.title("Titanic Correlation Heatmap")
plt.show()
```

Look for strong correlations with `survived`:

Discussion: Which factors most affected survival?

We Do: Subsetting a Heatmap

Full heatmaps with many columns are hard to read. Subset them:

```
# Focus on what correlates with survival
subset = corr[["survived"]].sort_values(
    by="survived", ascending=False)

plt.figure(figsize=(4, 6))
sns.heatmap(subset, annot=True, cmap="coolwarm", fmt=".2f")
plt.title("Factors Correlated with Survival")
plt.show()
```

This single-column heatmap makes the story clear:

- Top = strongest positive correlation
- Bottom = strongest negative correlation

We Do: Choosing the Right Chart

Question You're Asking	Best Chart
How is a value distributed?	**Histogram**
How does something change over time?	**Line plot**
How do categories compare?	**Bar plot**
What's correlated with what?	**Heatmap**
How do multiple variables relate?	**Pair plot**
How does a group differ from another?	**Box plot**

> The chart type should match the **question**, not the data.

You Do: Create a Chart (5 min)

Using the Titanic data (or any DataFrame you have):

Pick ONE of these and write the code:

1. A **bar plot** showing average fare by passenger class
   Hint: `titanic.groupby("pclass")["fare"].mean()`
2. A **histogram** of fares paid
   Hint: `plt.hist(titanic["fare"], bins=30)`
3. A **line plot** of average age by passenger class
   Hint: `titanic.groupby("pclass")["age"].mean()`

Don't forget: title, xlabel, ylabel!

Assignment Preview

This week's Kaggle notebook uses the **Diabetes Health Indicators** dataset:

Task	What You'll Do
1–2	Load data and understand column meanings
3	Histogram of age distributions
4	Line plot: health by age group
5	Full correlation heatmap
6	Subset heatmaps: top/bottom correlators
7	Pair plot: BMI vs Age by diabetes status
8	**Add visualizations to your capstone!**

Assignment Tips

Task 4 (line plot): Health scores are 1–5 where 5 = worst. Create a "Health" column = `5 - GenHlth` so higher = better.

Task 6 (subset heatmaps): Sort the correlation matrix by the column you care about, then plot just the top 10 and bottom 10 rows.

Task 7 (pair plot): BMI has too many unique values. Use `pd.qcut()` to bucket it into 10 quantiles for a cleaner chart.

Task 8 (capstone): Add 3+ charts to your capstone notebook. This completes your data pipeline project!

Capstone Visualization Goals

Your **Kaggle Data Pipeline** capstone should now include:

- Data loading and inspection (Lesson 4)
- Data cleaning and validation (Lesson 5)
- Wrangling, aggregation, and features (Lesson 6)
- **3+ visualizations with explanations** (this week!)

> After this week, your first capstone project is feature-complete. You can keep refining it until the end of the course.

Wrap-Up

Today we covered:

- Matplotlib: line plots, bar plots, histograms
- Customization: titles, labels, grids, legends, colors
- Seaborn: heatmaps and pair plots
- Subplots for multiple charts in one figure
- Choosing the right chart for the right question

Quick check: If you want to see how two variables relate to each other, which chart type would you use?

Getting Help This Week

- **Understand the data first** — read the column descriptions before plotting
- Experiment in a Kaggle notebook — you can see charts inline
- Matplotlib docs: matplotlib.org/stable/gallery
- Seaborn docs: seaborn.pydata.org/examples
- Post questions in **#discussion** on Slack
- Book a **1:1 mentor session** for help with your capstone charts

Next week: Introduction to Web Scraping — pulling data from the web!

Let Your Data Speak! 📈

A good chart tells a story that a spreadsheet never could.

See you next session!